

Let's break down the recurrence relation step-by-step to see exactly how fast this algorithm blows up.

1. Setting Up the Recurrence Relation

Let $T(n)$ be the worst-case time taken by the function $f(s, t)$ where n is the length of the strings.

In the worst-case scenario, the algorithm evaluates every single condition without short-circuiting early. This happens when the strings are *almost* scrambles, forcing the `||` and `&&` operators to check both the unswapped and swapped branches.

For a specific split at index c :

- **Unswapped check:** Makes two recursive calls: $f(s[0:c], t[0:c])$ which takes $T(c)$ time, and $f(s[c:n], t[c:n])$ which takes $T(n - c)$ time.
- **Swapped check:** Makes two more recursive calls: $f(s[0:c], t[n-c:n])$ which takes $T(c)$ time, and $f(s[c:n], t[0:n-c])$ which takes $T(n - c)$ time.

For any given split c , the total recursive work is:

$$2T(c) + 2T(n - c)$$

Additionally, at each split, the `substr()` operations and string comparisons (`s == t`) take $\mathcal{O}(n)$ time. Since the loop runs from $c = 1$ to $n - 1$, the total non-recursive work across the entire loop is $\mathcal{O}(n^2)$.

2. Formulating the Summation

To find the total time $T(n)$, we sum the work done over all possible split points from $c = 1$ to $n - 1$:

$$T(n) = \sum_{c=1}^{n-1} (2T(c) + 2T(n - c)) + \mathcal{O}(n^2)$$

Because summing $T(n - c)$ from 1 to $n - 1$ yields the exact same terms as summing $T(c)$ from 1 to $n - 1$ (just in reverse order), we can combine them:

$$T(n) = 4 \sum_{c=1}^{n-1} T(c) + \mathcal{O}(n^2)$$

3. Solving the Recurrence

To solve this, we can use the subtraction method. Let's write out the equation for n and $n - 1$.

Equation 1 (for n):

$$T(n) = 4T(1) + 4T(2) + \cdots + 4T(n-2) + 4T(n-1) + \mathcal{O}(n^2)$$

Equation 2 (for $n - 1$):

$$T(n-1) = 4T(1) + 4T(2) + \cdots + 4T(n-2) + \mathcal{O}((n-1)^2)$$

Now, subtract **Equation 2** from **Equation 1**:

$$T(n) - T(n-1) = 4T(n-1) + \mathcal{O}(n)$$

(Note: $n^2 - (n-1)^2$ simplifies to $2n-1$, which is $\mathcal{O}(n)$)

Rearranging the terms to isolate $T(n)$:

$$T(n) = 5T(n-1) + \mathcal{O}(n)$$

4. The Final Time Complexity

The relation $T(n) = 5T(n-1)$ is a linear homogeneous recurrence. By repeatedly expanding it:

- $T(n) \approx 5 \cdot (5T(n-2))$
- $T(n) \approx 25T(n-2)$
- $T(n) \approx 5^k T(n-k)$

This clearly forms a geometric progression. The dominant term is 5^n .

Therefore, the worst-case Time Complexity is exactly:

$$\mathcal{O}(5^n)$$

Space Complexity: The maximum depth of the recursion tree is n . At each level, we are creating substrings which take $\mathcal{O}(n)$ space. Therefore, the overall Auxiliary Space Complexity is $\mathcal{O}(n^2)$.

This $\mathcal{O}(5^n)$ complexity confirms why a string of just length 30 would require roughly 9×10^{20} operations, completely freezing the execution.